

MTM4692– APPLIED SQL

CONSTRUX: INTEGRATED SITE LOGISTICS & RESOURCE MANAGEMENT SYSTEM

GihubRepo: <https://github.com/coduronin/ConstruX-Logistics-System/>

PREPARED BY:

Tural Mammadli

Ceydanur Akdoğan

Merve Nur Çopuroğlu

Izzat Mammadzada

ISTANBUL, 2026

1. Introduction

This project, Construx, was built to handle the logistics and resource management for multiple construction sites. The main focus is on creating a working relational database that tracks how workers, heavy machinery, and materials move between different projects. Instead of using separate logs or spreadsheets that don't talk to each other, this system puts everything into one place so site managers can see exactly what is happening at any location in real-time. This report explains how the database is structured, why this model is necessary for site efficiency, and how we used SQL features like Joins and Views to pull useful reports.

2. Problem Statement

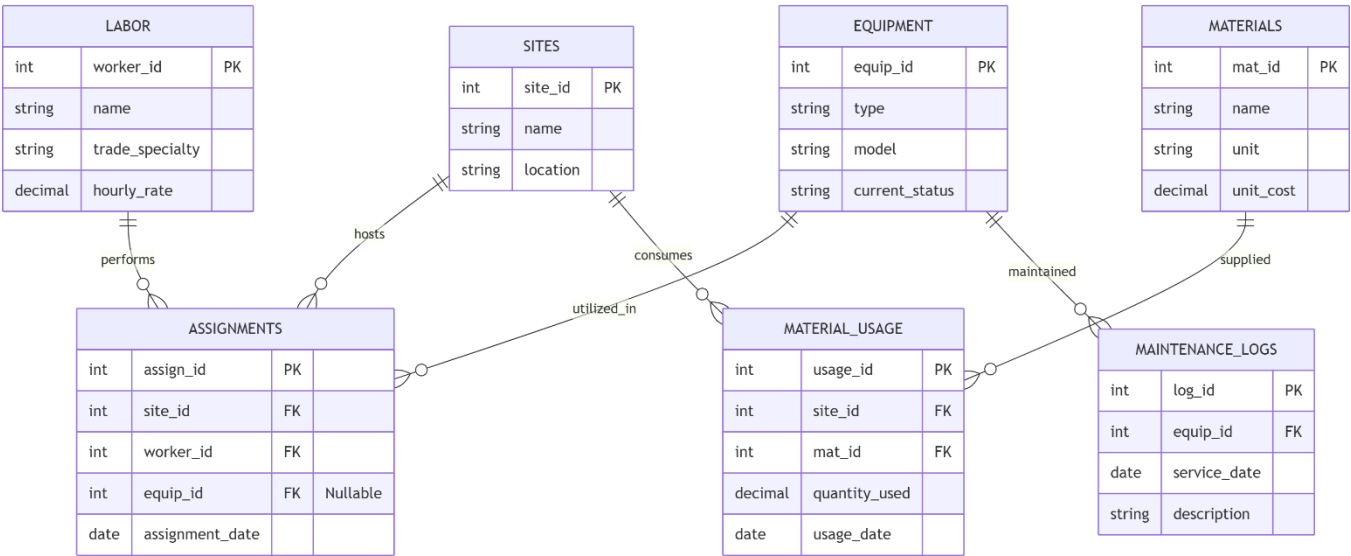
The real-world problem Construx solves is the lack of coordination on large construction sites. Currently, most sites manage their data in "silos"—maintenance schedules are in one place, worker shifts in another, and material orders in a third. This leads to serious errors. For example, a site manager might assign a crane operator to a site, but the crane itself is actually down for maintenance. Or, a team shows up to pour a foundation but there is no record of the concrete delivery. By centralizing sites, labor, equipment, and materials into one relational system, we eliminate these scheduling conflicts. The database makes sure that every assignment is backed by a machine that is actually "Active" and materials that are actually in stock.

3. Why It Matters

In civil engineering and construction, the biggest cost is lost time. This project is important because it moves site management away from manual tracking, which is prone to human error. Using a relational database allows us to enforce data integrity through foreign keys. This means the system physically prevents mistakes, like assigning a worker to a site that doesn't exist or double-booking expensive machinery. Furthermore, the reporting power of SQL allows for instant financial tracking. Instead of manually calculating costs at the end of the month, we can run a single query to see the total material spend for any site in seconds. This prevents budget overruns and keeps the project on the "critical path" for completion.

4. Preliminary Schema

The database consists of seven tables. This architecture was designed to be highly normalized, meaning we store data once and reference it everywhere else to avoid duplication errors.



Data Dictionary and Table Relationships:

Table Name	Primary Key	Foreign Keys	Purpose
Sites	site_id	None	Stores active project names and physical locations.
Labor	worker_id	None	Stores worker names, hourly rates, and trade specialties (e.g., Crane Operator).
Equipment	equip_id	None	Inventories heavy machinery and tracks their current status (Active/Idle/Maintenance).
Materials	mat_id	None	Catalog of consumables like C30 Concrete and Rebar with unit costs.
Assignments	assign_id	site_id, worker_id, equip_id	Links workers and machines to specific sites on specific dates.
Material_Usage	usage_id	site_id, mat_id	Tracks exactly how much of each material is used at each site.
Maintenance_Logs	log_id	equip_id	Maintains a detailed repair and service history for all machinery.

The relationships are strictly enforced. For example, the Assignments table acts as a junction between Sites, Labor, and Equipment. We made the equipment ID nullable here because some workers, like general laborers, do not operate heavy machinery. The Material_Usage table connects Materials to Sites, allowing us to track consumption per project.

5. Proposed SQL Features

To make the data useful for a site manager or accountant, we used several key SQL techniques:

- **Multi-Table Joins:** We used 4-way joins to connect the assignments with site names, worker names, and equipment types. We used LEFT JOINS for the equipment column so that if a worker doesn't have a machine, they still appear in the labor report instead of being deleted from the results.
- **Aggregation and Math:** We used the SUM() function combined with mathematical operators (quantity * cost) to calculate total site spending. This allows the database to do all the accounting work automatically.
- **Database Views:** We created four Views (site_overview, site_material_cost, equipment_maintenance_history , and idle_equipment). These are virtual tables that store the logic of our most important queries so that a user can see a high-level summary without needing to write complex code themselves.
- **Check Constraints:** We implemented constraints on the hourly rate and material cost columns to ensure no negative numbers are entered, protecting the financial accuracy of the database.
- **Foreign Key Constraints:** We used foreign key relationships to maintain referential integrity between tables. Additionally, we implemented ON DELETE CASCADE in the maintenance_logs table so that when an equipment record is deleted, all related maintenance logs are automatically removed. This prevents orphan records and keeps the database consistent.

6. Conclusion

The Construx project provides a reliable framework for managing construction logistics. By using a relational model, we have created a system that is far more accurate and scalable than traditional spreadsheets. The use of automated queries and views ensures that site managers have the information they need to keep projects moving without delays or budget errors.